

ARC-AGI-3 · 25-GAME PUBLIC EVALUATION · NO TRAINING, NO DEMONSTRATIONS

OPINE-World: **programmatic world models**, learned online from raw interaction

A single scored run per game. Two cooperating LLM agents play an unknown game from raw 64×64 frames, write its dynamics as a Python program, admit the program only when it replays all observed transitions exactly, and steer exploration by a Bayesian measure of object-type adequacy we call ontology error.

David Curtis · Wenhao Li · Scott Sanner · University of Toronto, [D3M Lab](#)

[arXiv 2607.01531](#) [PDF](#) [Code](#) [Reproduce](#)

20/25

games won, of 25 in the public evaluation set

78.4

mean action-efficiency score

1.7×

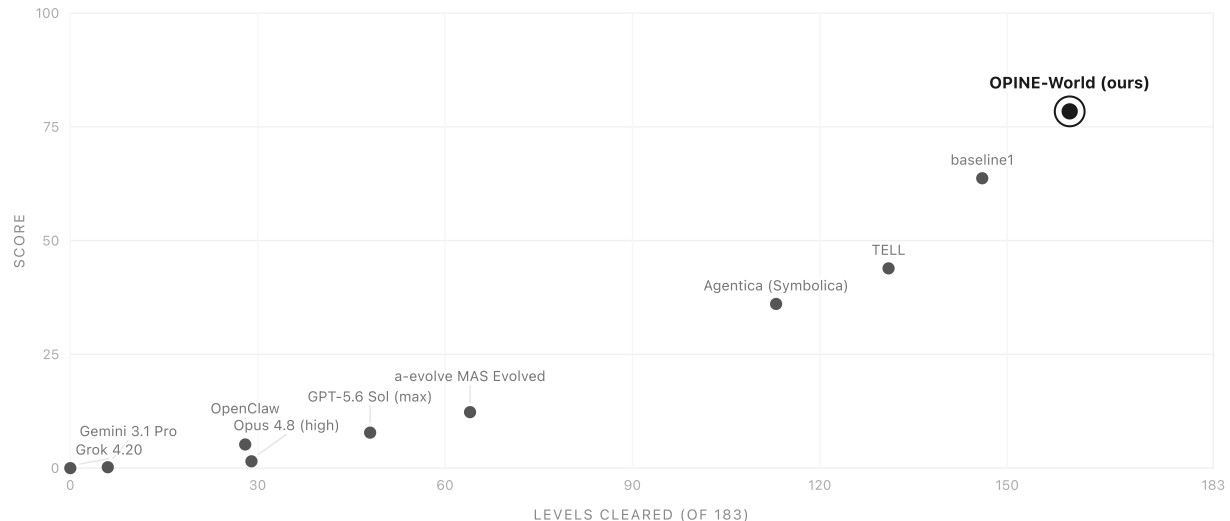
human pace, averaged over the 20 wins

0

gradient updates, demonstrations, or per-game engineering

RESULTS · score vs levels cleared

ARC-AGI-3 scores a run by how few actions it spends against a per-level human baseline, so the score rewards clearing games **and** clearing them efficiently. OPINE-World reaches a mean score of **78.4** and wins **20 of 25** games.



Frontier-LLM baselines are the official single-run sessions published per task at arcprize.org (fetched 2026-07-11, mirrored in `assets/arcprize_baselines.json`).

METHOD · hypothesis, test, repair

An object-centric world model, **synthesized as code, verified by exact replay**

01 · THE LOOP

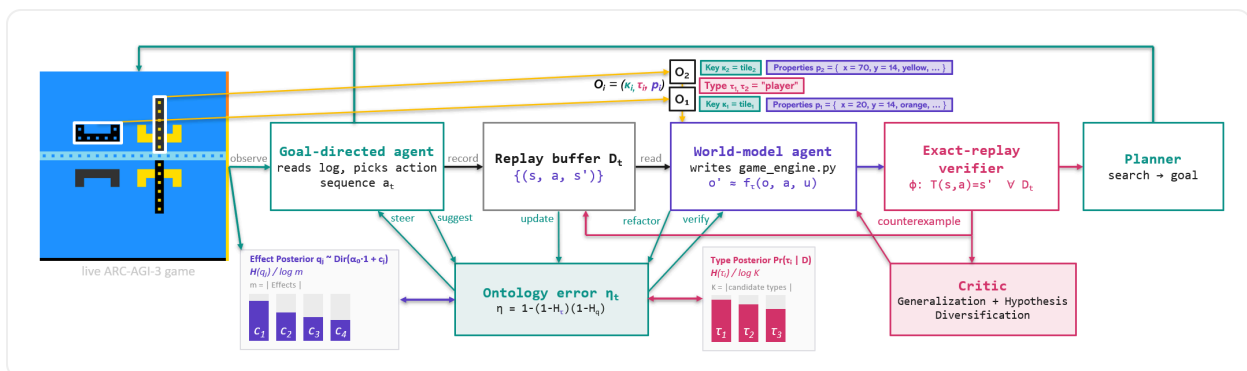
Two agents, a shared replay buffer, an exact-replay verifier

Game transitions are recorded into a buffer $D_t = \{(s, a, s')\}$. A goal-directed agent reads the log and picks the next action sequence. A world-model agent reads the same buffer and writes the dynamics as a Python program: $o' \approx f_\tau(o, a, u)$, one rule per type τ in local context u . A candidate engine $\hat{T}$ is admitted when

$$\hat{T}(s, a) = s' \text{ for every } (s, a, s') \in D_t$$

acceptance is exact replay of the buffer, a misprediction is a counterexample and the trigger for a rewrite

A rejected proposal's mispredicted transition becomes the counterexample for the next round. After the first level clear, a planner searches the verified program, validating planned steps against the live game.



02 · OBJECTS AND EFFECTS

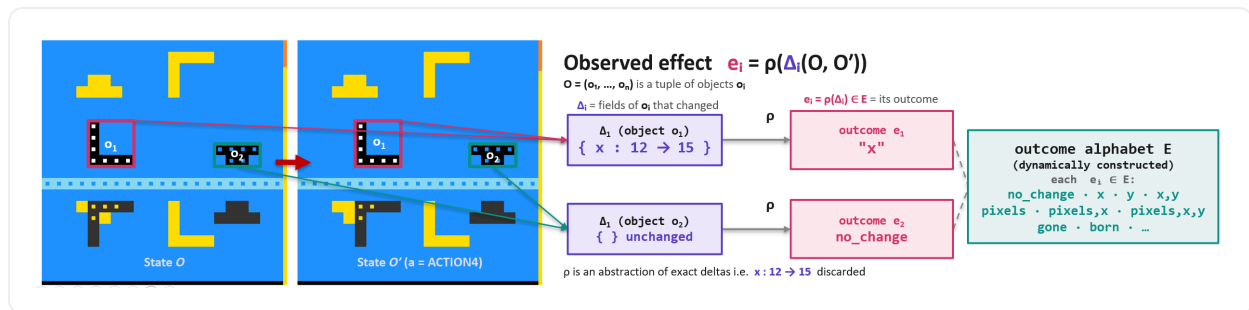
States are objects, changes are abstract effects

A state is a tuple of objects $O = (o_1, \dots, o_n)$, and each object $o_i = (\kappa_i, \tau_i, p_i)$ carries a tracking key κ_i , an inferred type τ_i , and properties p_i (position, pixel pattern). The extractor producing these is itself synthesized from the raw 64×64 grids. Under action a , the changed fields of o_i form its delta Δ_i , and the observed effect keeps the shape of the change

$$e_i = \rho(\Delta_i(O, O'))$$

ρ discards magnitudes, $x : 12 \rightarrow 15$ becomes "x", making outcomes comparable across objects and levels

The outcome alphabet E grows from observation: no_change · x · y · pixels · gone · born. A few transitions suffice to show that two objects respond alike, the signal the type layer uses.



03 · ROWS AND CONTEXTS

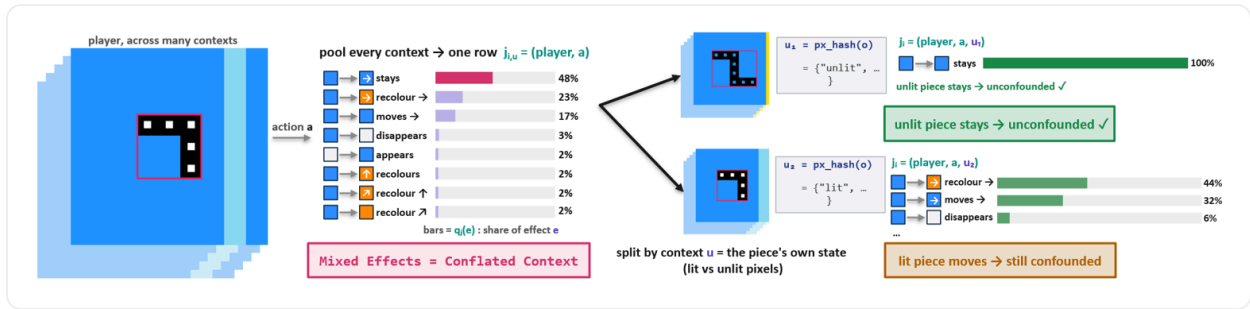
Effect rows keyed by type, action, and local context

Object-transitions are filed into rows keyed by type, action, and local context, $j = (\tau, a, u)$. Row counts $c_j(e)$ with a symmetric Dirichlet prior give a posterior over the row's effect distribution

$$q_j \sim \text{Dir}(\alpha_0 \cdot \mathbf{1} + c_j)$$

α_0 is a small pseudocount per effect, sparse rows remain uncertain, and the posterior is closed form

A row that remains mixed after many samples indicates a conflated context: a piece both "stays" and "recolours" under one action because a hidden condition, its lit or unlit pixel state, was pooled away. Splitting on that context u separates the mixture into near-pure rows, implementable as per-type rules f_τ . The Dirichlet form gives calibrated uncertainty from small counts without fitting.



04 · ONTOLOGY ERROR

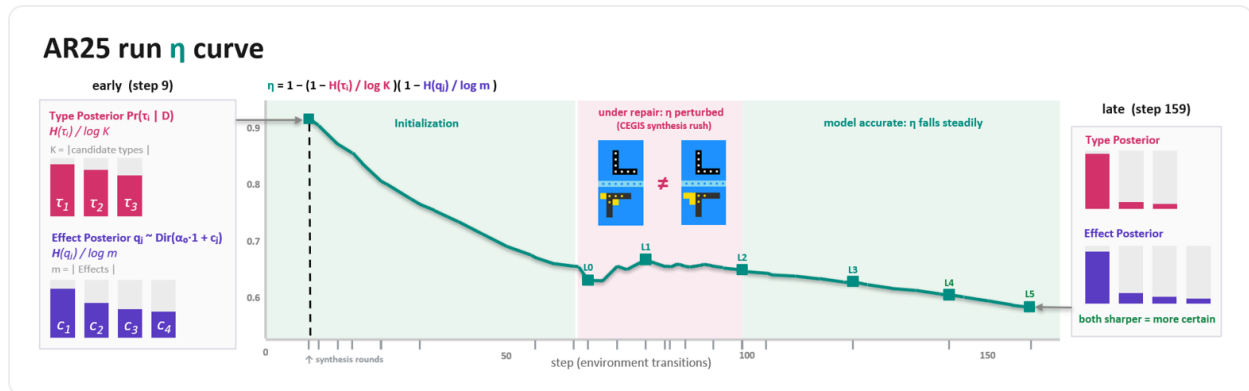
Ontology error η : joint type and effect uncertainty

Two normalized entropies measure what is unresolved: type uncertainty $H(\tau_i) / \log K$ from the posterior $\Pr(\tau_i | D)$ over K candidate types, and effect uncertainty $H(q_j) / \log m$, with m the number of observed effects. Both lie in $[0, 1]$. Ontology error combines them

$$\eta = 1 - \left(1 - \frac{H(\tau_i)}{\log K}\right) \left(1 - \frac{H(q_j)}{\log m}\right)$$

the product form drives η to 0 when both type and effects are resolved, and either uncertainty keeps it high

Exploration is prioritized toward high- η rows, where the model mispredicts, rather than re-deriving known dynamics. Over a run the curve falls as posteriors sharpen, is perturbed during CEGIS repair, and declines once the engine replays the buffer. The annotated curve below is the ar25 run.

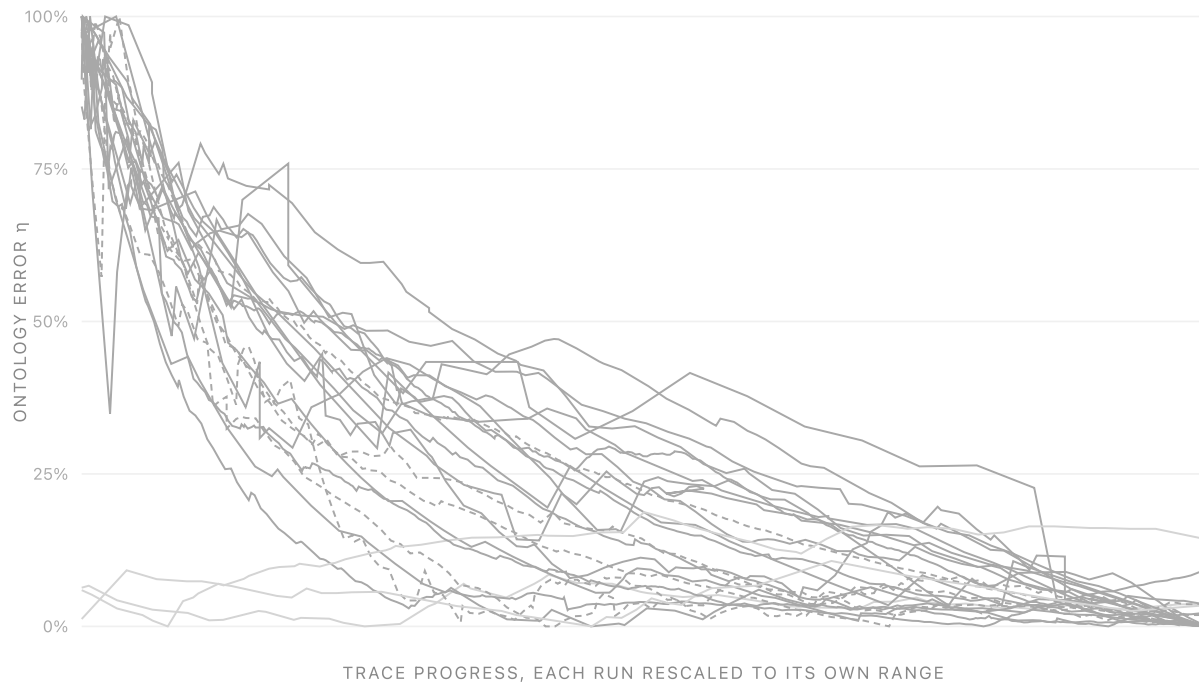


OBSERVATIONS · what the run artifacts show

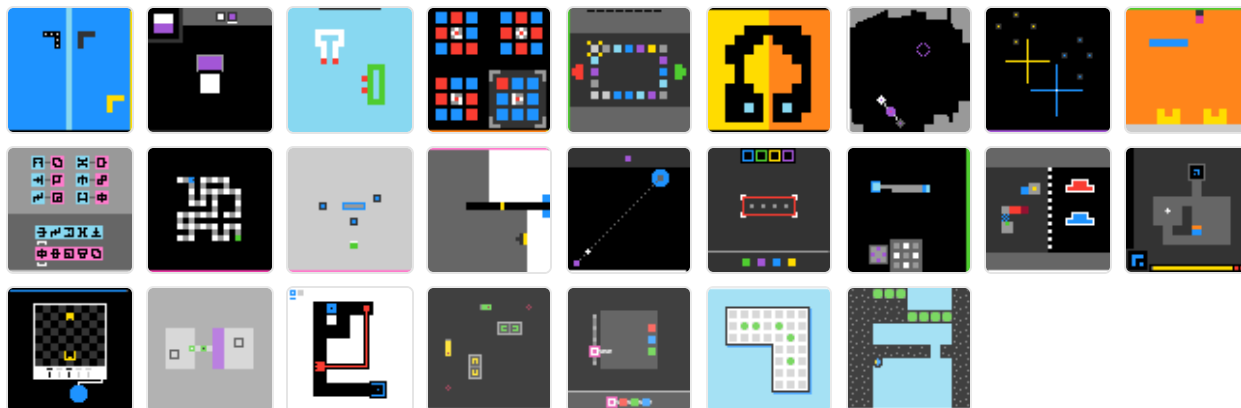
01 · η ACROSS ALL 25 GAMES

η trajectories across the 25 runs

One line per run, rescaled to its own η range and trace span, so the graph shows trajectory shape. Hovering a game in the selector (tapping, on a touch screen) transitions the chart to absolute coordinates with level breakpoints, synthesis events, and the run's ending. Three runs (ar25, tr87, vc33) recorded η from a late-engaging extractor onward, after the ontology had settled, and are drawn faded.



HOVER A GAME



— won - - - - errored ◇ level cleared, η valued (on hover) synthesis event (on hover) hover = absolute η , rest = relative shape

02 · HYPOTHESIS → CODE

Four traced hypothesis-test episodes

Each case is reconstructed from the released artifacts: the acting agent states a mechanism hypothesis (quoted verbatim from its decision log),

probes it in the live game, a synthesis round turns it into engine code, and the exact-replay verifier reports the accuracy. Clicking a case opens that moment in the replay.

01 · SP80 Repairing a falsified win condition	02 · VC33 Finding a mechanic missing from the model	03 · DC22 Forcing a repair, then executing a model-checked route	04 · SC25 Testing a mechanic carried over from level 1
---	---	--	--

sp80 · step 83

► Watch this moment

Repairing a falsified win condition

*"Real fire74 filled all 3 cups yet scored 0 => win is strict
terminals==openings, not 'all filled'. Forcing CEGIS with concrete terminal data + cup-wall/center-pass-through questions, and RESET to a clean board to replan with the repaired model."*

goal-directed agent ·
call_0024 · step 83

probe **RESET** →

synthesis round **9** @ step 83

→

exact replay **63 → 84**

transitions @ 100%

→ η **0.73 → 0.70**

*"barY@0-23 blocks
beamA with its LEFT pour
going OFF-BOARD (no 0-
3 stray) and right pour 24-
27 -> barW@12-27 -> 8-
11(cup1)+28-31(cup2);
barX@32-51 -> beamB ->
28-31+52-55(cup3);
paddle@40-63 -> beamC
off-board. Synth-verified
on-board terminals ==
openings"*

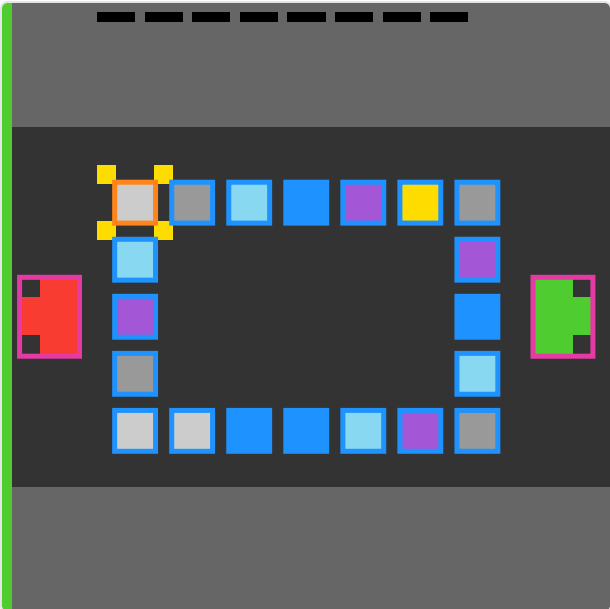
confirmation · step 89

A fire at step 74 filled the cups and scored zero. The acting agent concluded the win predicate itself was wrong, forced a CEGIS round with the divergent transition as a counterexample, and reset the board. The repaired engine verified a configuration whose beam terminals exactly equal the cup openings, and the fire at step 95 cleared the level.

Inside the final lp85 engine

Let's see the LP85 game engine: this game used the planner to clear levels 3, 4 and 6, four other plans were aborted mid-flight by the per-step validation guard, and the game won in 109 actions, 3.5 times the human pace. The final engine is 1,116 lines, its four public functions are short, and the mechanics are generalized into helpers for rotors, cycles and cascades.

The extractor emits three types: block, target (a block that currently sits under the cursor pair), and button (a rotation cross whose direction is read from its color). One rotation rule, `_apply_rotation`, describe the dynamics. The planner compiles each button into a permutation over its rotor cells and searches token positions, keeping deep routings inside the search budget.

OBJECT EXTRACTION	TRANSITION AND REWARD FUNCTIONS	PLANNER: COMPILING CLICKS TO PERMUTATIONS	PLANNER: BFS OVER TOKEN POSITIONS
 step 0, the L1 entry. 19 blocks, one cursor-slot target, two rotor buttons, typed by this extractor on this frame. target block button			

```

① def extract_objects(frame):
    grid = [[int(v) for v in row
    if not grid or not grid[0]:
        return []
    objects = []
    blocks = _square_blocks(grid
    slots = set(id(s) for s in _
    ②     for i, b in enumerate(bloc
        bw, bh = b["w"], b["h"]
        pixels = [[-1] * bw for
        for (x, y) in b["cells"]
            pixels[y - b["top"]]
    ③     objects.append({
        "name": f"block_{i:0
        "type": "target" if
        "tags": [f"color_{b[
        + (["cursor_slot"] i
        "x": b["left"], "y":
        "pixels": pixels, "l
    })
    ④     for i, c in enumerate(_cro
        objects.append({
            "name": f"cross_{i:0
            "type": "button",
            "tags": ["rotate_ccw
            "x": c["left"], "y":
            "layer": 0, "visible
        })
    ⑤     return objects

```

lp85 · synthesis/run_022/game_engine.py · L1088–L1116 · comments stripped

① Blocks and slots from components

Square lattice blocks come from connectivity, and the cursor pair marks the goal slots.

② Pixel crop per block

Each object carries its own sprite crop, so downstream rules and the verifier read object dicts.

③ Slot state is a type switch

A block is typed target while it is a cursor slot. Color and slot membership travel as tags.

④ Buttons with direction from color

Crosses are the clickable rotors. Red means counter-clockwise, green clockwise, learned from observed rotations.

⑤ Three types, eight levels

block, target, button. One extractor parses all levels of the run.

04 · CONTEXT ECONOMICS

Token accounting for the two-agent split

Each CEGIS round runs a fresh synthesizer session: it reads the buffer and shared docs, rewrites `game_engine.py`, reports replay accuracy, and exits. The numbers below are summed from per-round API accounting.jsonl.

863

synthesis rounds across the 25 runs, one fresh session per round

35.9M

tokens of engine code and synthesis reasoning written by the world-model agent

0

of those tokens are forced into acting agent's context unless explicitly requested by the agent

A round consumed 10.5M prompt tokens on average across its turns (median 8.7M, p90 18.9M) and took 12.1 minutes of wall clock, 174 hours in total. The acting agent runs against a context compacted at roughly 500k tokens, one round of synthesis material would not fit beside its working view of the game. The agents share artifacts through files that can be explicitly read.

REPLAYS · all 25 scored runs, from the released artifacts

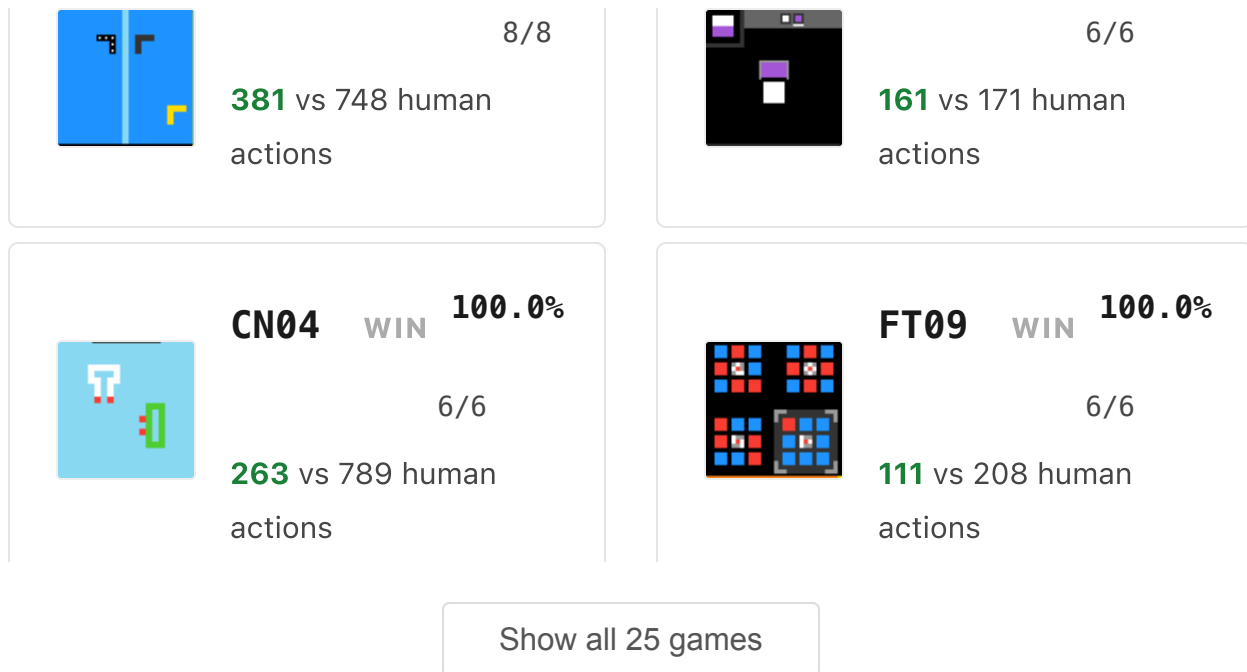
Replays of all 25 scored runs, **with per-step reasoning and engine state**

Each card replays the scored run, including intermediate animation frames. The **Reasoning** tab follows the playhead and shows the move set the acting agent chose at that step, with its rationale. The **Engine** tab shows the synthesized world model as of that step. Diamonds on the timeline mark synthesis rounds, and added lines are highlighted against the previous round.

Click a game to open its replay · dots are levels: filled = at or above 50% level score, gray = cleared slower, hollow = budget ended there

AR25 WIN **100.0%**

CD82 WIN **100.0%**

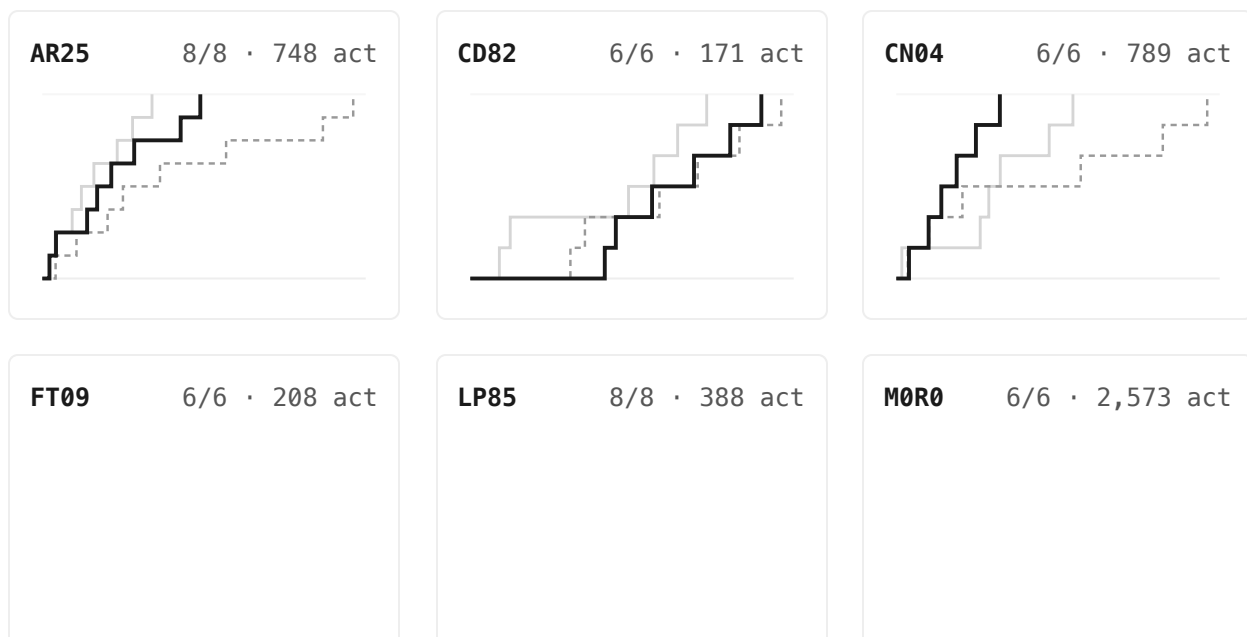


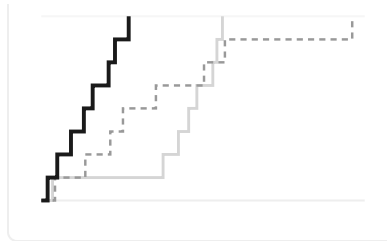
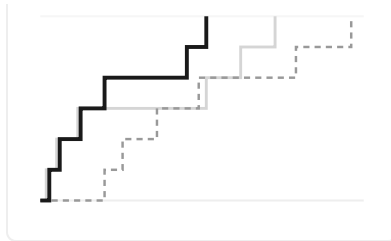
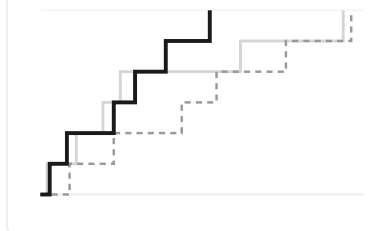
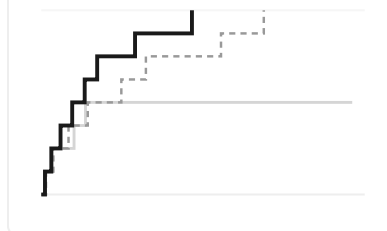
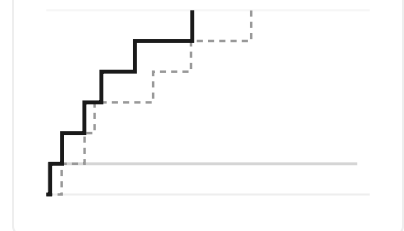
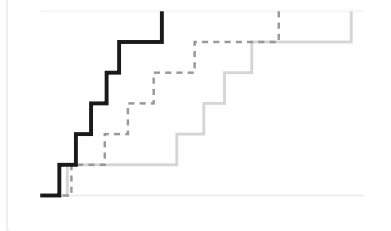
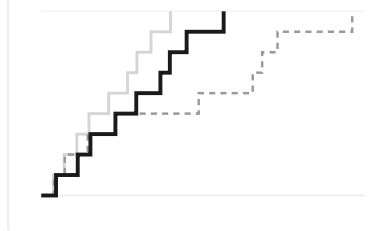
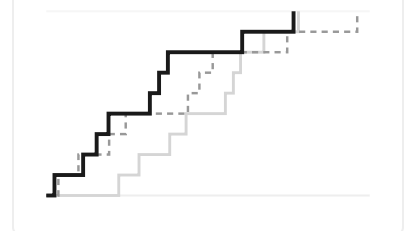
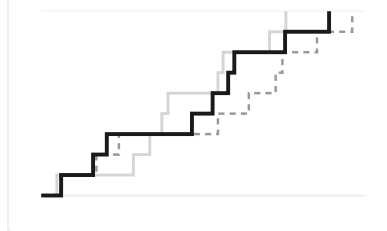
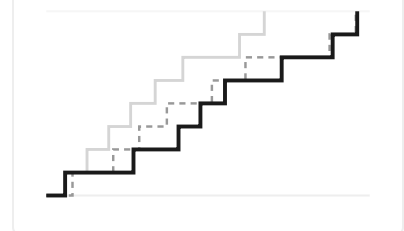
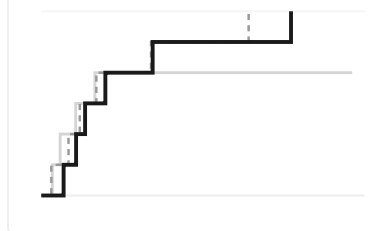
LEVELS VS ACTIONS

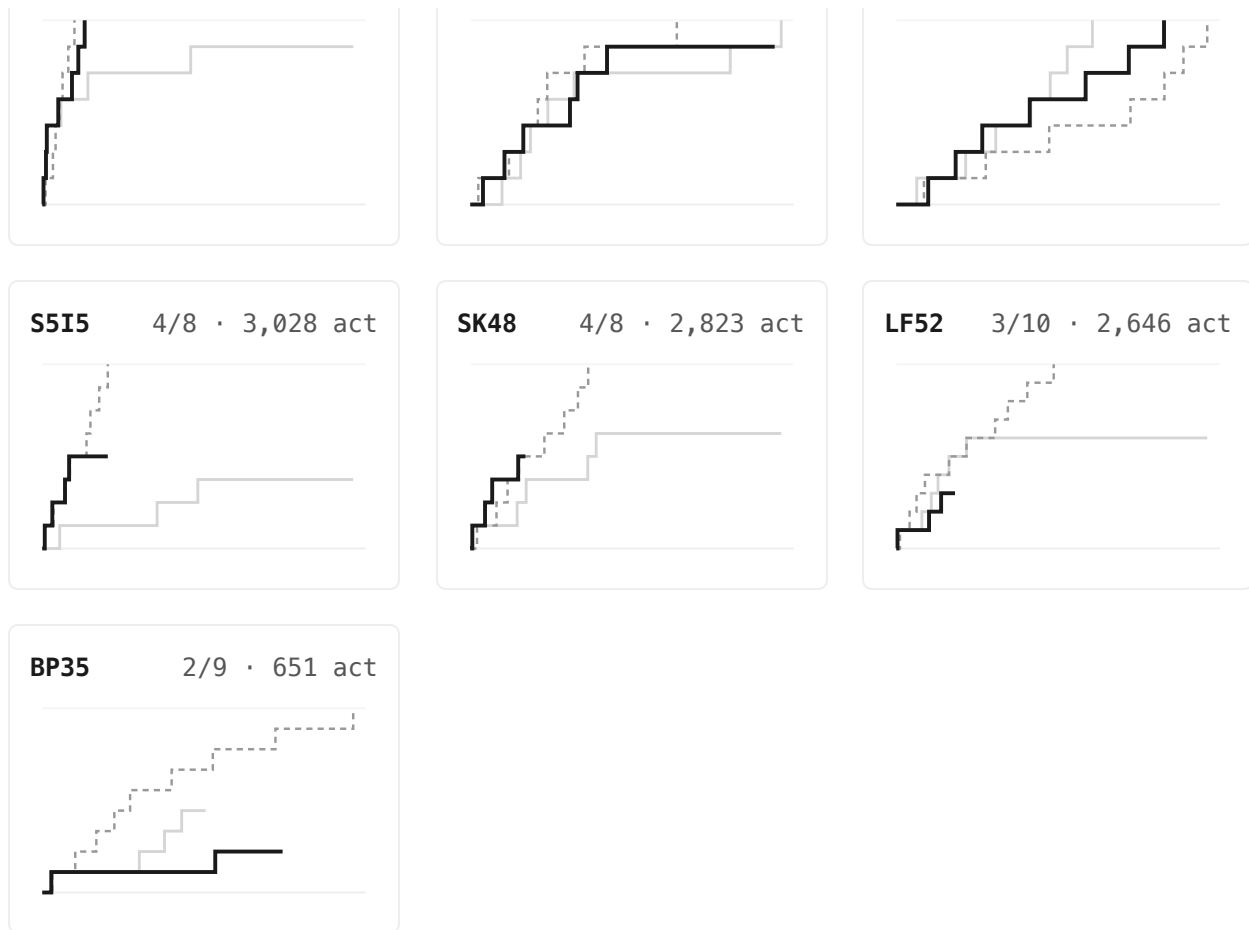
Levels cleared against cumulative actions, per game

Levels cleared against cumulative actions, as on the official task pages. Black is OPINE-World, gray the human reference, light gray baseline1. A flat tail is budget spent on an uncleared level. Clicking a game opens its replay.

— OPINE-World - - - - human — baseline1

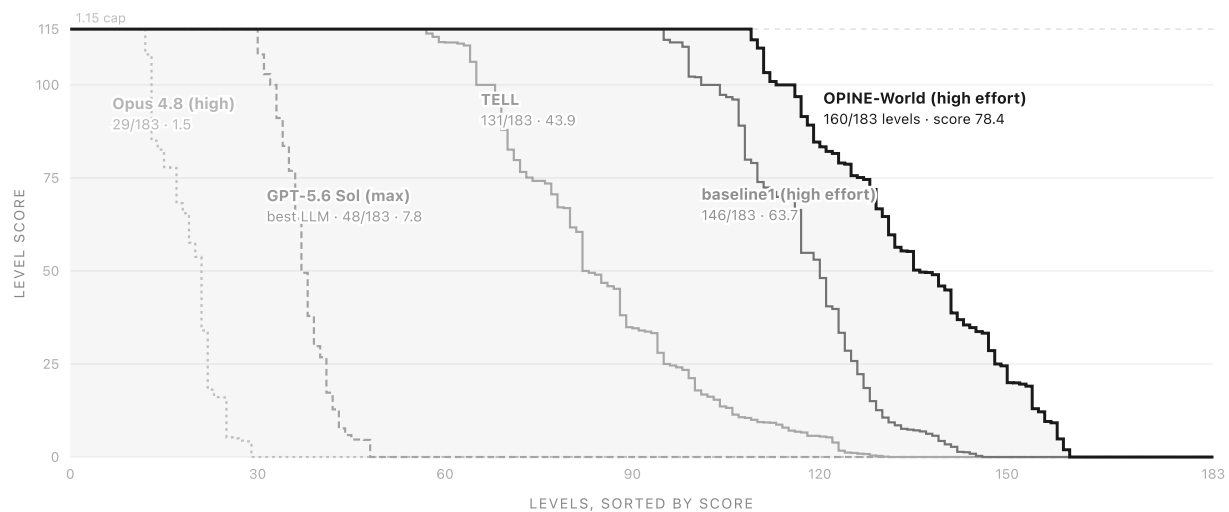


**R11L** 6/6 · 233 act**RE86** 8/8 · 1,754 act**SP80** 6/6 · 786 act**TR87** 6/6 · 540 act**TU93** 9/9 · 462 act**WA30** 9/9 · 1,843 act**VC33** 7/7 · 1,596 act**SU15** 9/9 · 361 act**SB26** 8/8 · 214 act**SC25** 6/6 · 1,107 act**DC22** 6/6 · 1,842 act**LS20** 7/7 · 959 act**TN36** 7/7 · 3,058 act**KA59** 6/7 · 1,099 act**G50T** 7/7 · 879 act



SCORE PER LEVEL

Per-level scores across all 183 levels



More area is better. A level scores $\min(1.15, (\text{human}/\text{agent})^2)$, where human and agent are the number of actions the human baseline and the agent took to clear the level; uncleared levels score 0. ARC-AGI-3 gives no extra credit for clearing a level in far

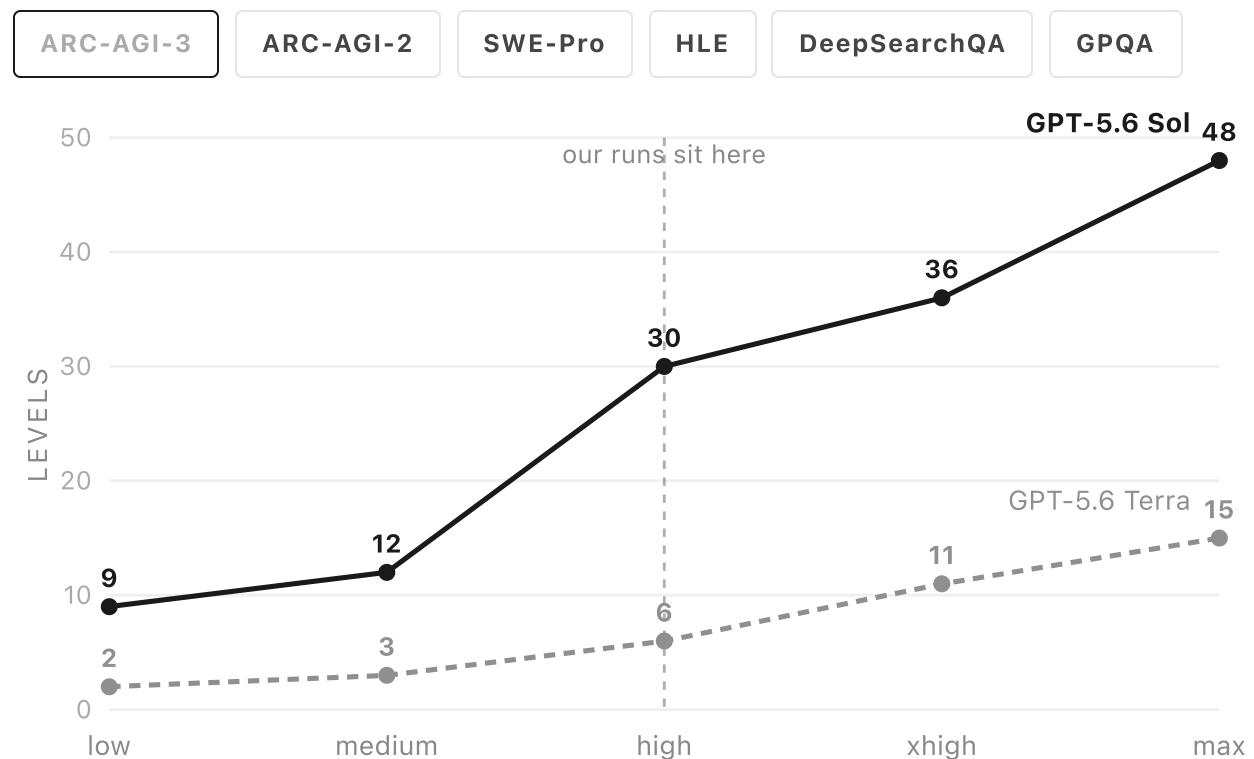
fewer actions than the human (1.15 cap): OPINE-World beats the cap on 109 of its 160 cleared levels, often by large margins the score cannot reflect.

05 · EFFORT HEADROOM · estimate

Room for increased effort

Both agents ran Opus 4.8 at high, the third of five reasoning-effort tiers. With xhigh and max efforts, gains are expected. We show this against the measured scores for the recent frontiers, as unfortunately, we are unable to report our results for higher-effort tiers due to financial constraints.

BUDGET · effort vs performance



Official per-task sessions, levels cleared across the 25 public games. Full ladders exist for the GPT-5.6 variants. Raw Opus 4.8's published session runs at high and clears 29 levels, at Sol's high (30).

Running Opus 4.8 on max is expected to deliver significant gains. Official per-task sessions at arcprize.org, levels cleared across the 25 games. Effort raises Sol from 9 to

48 levels, with the top two tiers adding roughly 60% over high. Anthropic recommends max for this kind of work ([announcement](#)).

ERRORS · hypothesizing recovered runs



25-GAME MEAN SCORE

Some promising runs that failed to win ran into API rate limits well before a 2000 move limit was reached. Due to the runtime and idle limitations of ARC-AGI-3's competition mode, this cut their solutions short. As an upper bound, if each errored game is set to the average score of the rest of the games, our final score would be 92.1. We expect our true score to reasonably lie between these ranges.

CITATION

```
@misc{courtis2026opineworldprogrammaticworldmodeling,
  title={OPINE-World: Programmatic World Modeling with
        Ontology-error-Prioritized Interactive Exploration for ARC-A},
  author={David Courtis and Wenhao Li and Scott Sanner},
  year={2026},
  eprint={2607.01531},
  archivePrefix={arXiv},
```

```
primaryClass={cs.AI},  
url={https://arxiv.org/abs/2607.01531},  
}
```

Replays, reasoning, and engine code are generated from the released per-game run artifacts by `docs/build_site_data.py` in the code repository. Scores use the ARC-AGI-3 action-efficiency metric against the human baseline. Page design adapted from [rednote-hilab/TELL](https://github.com/rednote-hilab/TELL).